

UNIT III – PROCESS SYNCHRONIZATION & DEADLOCKS

Process Synchronization - The critical-section problem, Semaphores, Monitors and Synchronization using Monitors Classic problems of synchronization, Critical regions, Monitors; Deadlock - System model, Deadlock characterization, Methods for handling deadlocks, Deadlock prevention, Deadlock avoidance, Deadlock detection, Recovery from deadlock.

PROCESS SYNCHRONIZATION:

Process Synchronization is defined as the process of sharing system resources by cooperating processes in such a way that, Concurrent access to shared data is handled thereby minimizing the chance of inconsistent data.

EXAMPLE:

- Consider the producer consumer process that contains a variable called counter.

Counter is incremented every time we add a new item to the buffer and is decremented every time we remove one item from the buffer.

- The code for the producer process is

```
while (true) {  
    /* produce an item in next produced */  
    while (counter == BUFFER SIZE)  
        ; /* do nothing */  
    buffer[in] = next produced;  
    in = (in + 1) % BUFFER SIZE;  
    counter++;  
}
```



The code for the consumer process is

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next consumed = buffer[out];  
    out = (out + 1) % BUFFER SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```

- Suppose that the value of the variable counter is currently 5 and that the producer and consumer processes concurrently execute the statements —counter++ and —counter--.

- Following the execution of these two statements, the value of the variable counter may be 4, 5, or 6!

- The only correct result, though, is counter == 5, which is generated correctly if the producer and consumer execute separately.

- When several processes access and manipulate the same data concurrently the outcome of the execution depends **on the particular order in which the access takes place, is called a race condition.**
- To guard against the race condition we need to ensure that only one process at a time can be manipulating the variable counter.

CRITICAL SECTION PROBLEM:

- ❖ The **critical-section problem** is to design a protocol that the processes can use to cooperate.
- ❖ Each process has a segment of code, called a **critical section**, in which the process may be changing common variables, updating a table, writing a file, and so on.
- ❖ When one process is executing in its critical section, no other process is allowed to execute in its critical section. That is, no two processes are executing in their critical sections at the same time.

The structure of critical section problem is

```
do {
    entry section
    critical section
    exit section
    remainder section
} while (true);
```

- ❖ Each process must request permission to enter its critical section. The section of code implementing this request is the **entry section**.
- ❖ The critical section may be followed by an **exit section**.
- ❖ The remaining code is the **remainder section**.

A solution to the critical-section problem must satisfy the following three requirements:

1. **Mutual exclusion.** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress.** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next.
3. **Bounded waiting.** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

SOLUTIONS TO CRITICAL SECTION PROBLEM:

PETERSON'S SOLUTION:

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections.

The processes are numbered P_0 and P_1 . For convenience, when presenting P_i , we use P_j to denote the other process.

Peterson's solution requires the two processes to share two data items:

int turn;

boolean flag[2];

The variable turn indicates whose turn it is to enter its critical section. That is, if $turn == i$, then process P_i is allowed to execute in its critical section.

The flag array is used to indicate if a process is ready to enter its critical section. For example, if $flag[i]$ is true, this value indicates that P_i is ready to enter its critical section.

```

do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

    critical section

    flag[i] = false;

    remainder section

} while (true);

```

To enter the critical section, process P_i first sets $flag[i]$ to be true and then sets $turn$ to the value j , thereby checking that if the other process wishes to enter the critical section, it can do so.

Similarly to enter the critical section, process P_j first sets $flag[j]$ to be true and then sets $turn$ to the value i , thereby checking that if the other process wishes to enter the critical section.

The solution is correct and thus provides the following.

1. Mutual exclusion is preserved.
2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.

MUTEX LOCKS:

Mutex locks are used to protect critical regions and thus prevent race conditions



A process must acquire the lock before entering a critical section; it releases the lock when it exits the critical section.

```

do {
    acquire lock

    critical section

    release lock

    remainder section

} while (true);

```

- The `acquire()` function acquires the lock, and the `release()` function releases the lock.
- A mutex lock has a boolean variable available whose value indicates if the lock is available or not.
- If the lock is available, a call to `acquire()` succeeds, and the lock is then considered unavailable.
- A process that attempts to acquire an unavailable lock is blocked until the lock is released.

The definition of `acquire()` is as follows:

```

acquire()
{
    while (!available)
    ; /* busy wait */
    available = false;;
}

```